



Jadavpur University

Department of Computer Science & Engineering

Flow Control Protocols: Stop-and-Wait, Go-Back-N ARQ & Selective Repeat ARQ

*A Hybrid C++/Python Network Socket Simulation with CRC-32
Frame Integrity, Fault Injection, and Sliding Window Performance Analysis*

Name:	Anubhab Rakshit
Class:	BCSE-III (Semester 5)
Roll Number:	002410501029
Assignment:	Lab Assignment 2
Submission Date:	September 2026

Contents

1 Purpose & Learning Objectives (CO2)	2
2 System Architecture & Workflow	2
2.1 Frame Construction & CRC-32 Pipeline	2
2.2 Unreliable Channel & Fault Injection	2
2.3 Receiver Validation & Reassembly	2
2.4 Protocol Transmission Ladder Diagrams	3
3 Project Structure	3
4 Frame Structure	4
5 Theoretical Modeling	4
5.1 Stop-and-Wait	4
5.2 Go-Back-N	4
5.3 Selective Repeat	5
6 Implementation Highlights	5
7 Test Methodology	5
8 Experimental Results	5
8.1 Throughput vs. Loss Rate	5
8.2 Retransmission Overhead	6
8.3 Latency Distribution	6
8.4 Window Size Scaling & Aggregated Metrics	6
9 Key Findings, Challenges & Conclusion	7

1 Purpose & Learning Objectives (CO2)

Flow control regulates the rate of frame transmission so a fast sender cannot overwhelm a slower receiver's finite buffers. This assignment implements and empirically compares the three canonical sliding-window ARQ paradigms — **Stop-and-Wait**, **Go-Back-N**, and **Selective Repeat** — fulfilling **CO2** (design and implementation of Logical Link Control flow-control mechanisms) over connectionless UDP sockets, without relying on TCP's built-in reliability. Key objectives:

- Bound frames-in-flight via a sliding window W to prevent receiver buffer overflow.
- Model an unreliable channel (loss, corruption, ACK loss) and quantify each protocol's resilience.
- Reuse the CRC-32 and framing primitives from Assignment 1 as a compiled C++ shared library, bound into the Python orchestration layer via a `ctypes` Foreign Function Interface (FFI) — mirroring real protocol stacks where hot-path primitives live in a systems language.

2 System Architecture & Workflow

2.1 Frame Construction & CRC-32 Pipeline

The input file is sliced into 46-byte payload chunks, prefixed with a 15-byte pseudo-MAC header (destination/source MAC, length, sequence number), converted to bits via the C++ FFI (`bytestobits()`), CRC-32 encoded (`encodecrc()`, IEEE 802.3 polynomial), and repacked into a 65-byte transmittable frame (`bitstowrite()`).

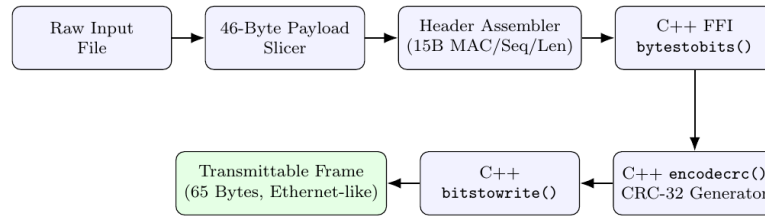


Figure 1: Sender Frame Construction Pipeline.

2.2 Unreliable Channel & Fault Injection

Four independent, independently-toggable fault mechanisms simulate a noisy link: random **packet loss** ($p_{\text{loss}} \in [0, 0.15]$), **bit inversion** (single/burst, reused from Assignment 1), **ACK dropping** on the reverse path, and **deterministic injection** (`--inject drop_X, corr_X`) for reproducible correctness tests.

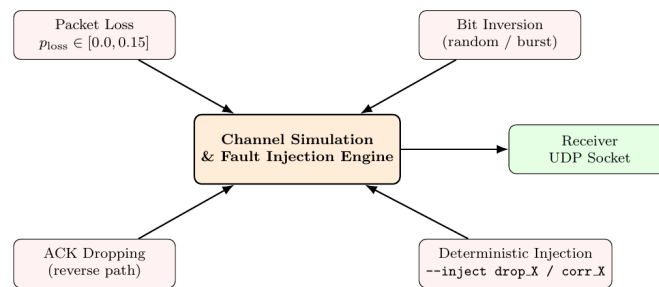


Figure 2: Channel Simulation & Error Injection Model.

2.3 Receiver Validation & Reassembly

`recvfrom()` hands each datagram to `crck_ok()` for independent CRC-32 recomputation; frames that fail are silently dropped (no NAK — the sender's timeout drives retransmission). Frames that pass are checked against a protocol-specific rule: Stop-and-Wait matches the alternating bit; Go-Back-N enforces strict in-order delivery ($W_R = 1$) with a cumulative ACK; Selective Repeat buffers

any frame in $[\text{base}, \text{base} + W - 1]$ in a hash-map and ACKs it individually, sliding **base** forward as it fills.

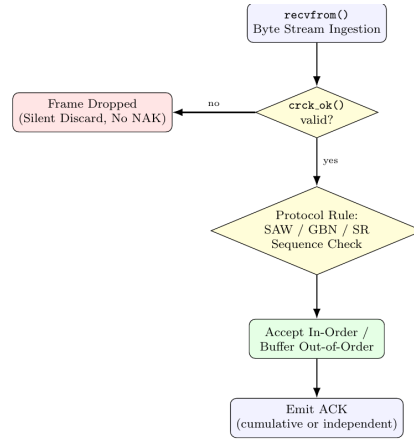


Figure 3: Receiver Frame Validation & Sliding Reassembly Pipeline.

2.4 Protocol Transmission Ladder Diagrams

Figure 4 contrasts all three protocols under an identical single-lost-frame fault, making their structural recovery difference explicit: Stop-and-Wait idles a full round trip; Go-Back-N discards and re-sends its entire trailing window; Selective Repeat buffers the undamaged frames and re-sends only the lost one.

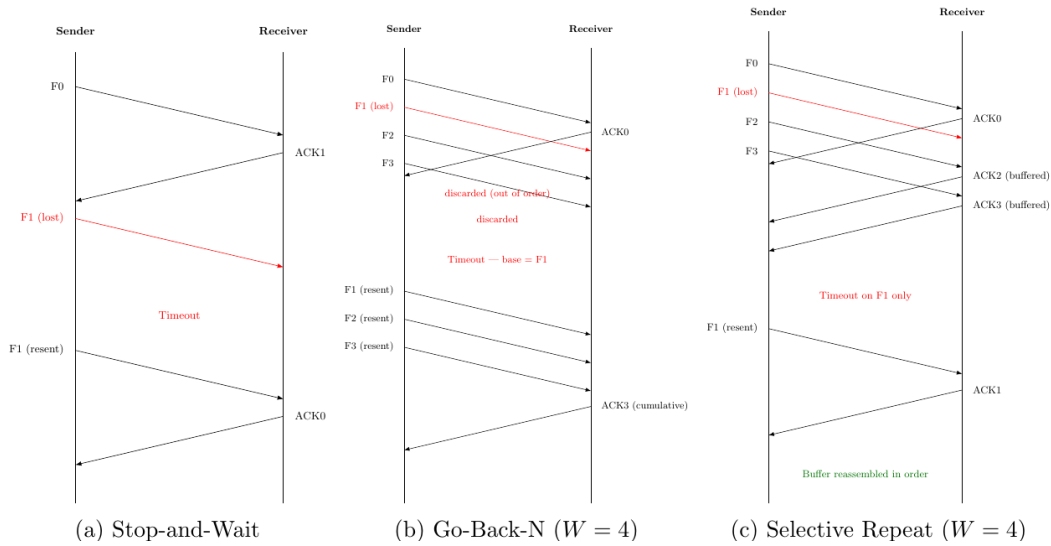


Figure 4: Protocol Transmission Ladder Diagrams under a single lost data frame.

3 Project Structure

A hybrid two-language architecture is used: bit-level primitives (CRC-32, byte/bit packing, error injection) live in compiled **C++**; window management, timers, retransmission and logging live in **Python** for rapid iteration.

The `ctypes` boundary in `utils.py` is the key architectural seam: buffers are marshalled via `create_string_buffer` into the C++ library and back, letting the CRC-32 polynomial division run at native speed while protocol logic stays in Python.

File	Responsibility
sender.py	SAW / GBN / SR sender engines, window state, timers, metrics logger
receiver.py	UDP server, frame parser, out-of-order reassembly (SR hash-map), ACK emitter
utils.py	ctypes FFI bindings exposing the compiled C++ library to Python
bridge.cpp/.so	Assignment-1 CRC-32 & framing functions behind an <code>extern "C"</code> ABI
simulate.py	Batch harness: 50 files $\times$ 17 configurations, subprocess orchestration
visualize.py	Aggregates CSV logs into the figures of Section 8 (Matplotlib/Seaborn)

Table 1: Project file structure and module responsibilities.

4 Frame Structure

Every frame follows a fixed 65-byte, Ethernet-II-like layout.

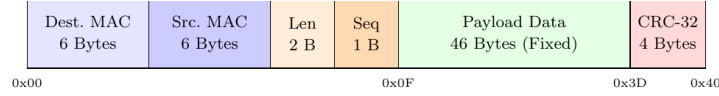


Figure 5: Byte-level layout of the 65-byte transmittable frame.

Field	Offset	Size
Destination MAC	0x00–0x05	6 B (constant 0xAA)
Source MAC	0x06–0x0B	6 B (constant 0xBB)
Payload Length	0x0C–0x0D	2 B, big-endian
Sequence Number	0x0E	1 B, modulo-256
Payload Data	0x0F–0x3C	46 B (fixed)
CRC-32 Trailer	0x3D–0x40	4 B, IEEE 802.3
Total		65 Bytes

Table 2: Byte-level frame field specification.

5 Theoretical Modeling

Let $T_{\text{trans}} = L/B$ be transmission time, T_{prop} one-way propagation delay, and $a = T_{\text{prop}}/T_{\text{trans}}$ the normalized delay (Bandwidth-Delay Product ratio).

5.1 Stop-and-Wait

The sender idles for a full round trip after each frame, giving ideal-channel efficiency

$$\eta_{\text{SAW}} = \frac{1}{1 + 2a}, \quad \eta_{\text{SAW,loss}} = \frac{1 - p}{1 + 2a}. \quad (1)$$

As $a \rightarrow \infty$ (high-BDP links), $\eta_{\text{SAW}} \rightarrow 0$: only one frame is ever in flight regardless of available bandwidth.

5.2 Go-Back-N

With window N and receiver window $W_R = 1$,

$$\eta_{\text{GBN}} = \min\left(1, \frac{N}{1 + 2a}\right), \quad \eta_{\text{GBN,loss}} = \frac{1 - p}{1 - p + Np} \cdot \frac{N}{1 + 2a}, \quad (2)$$

since a single loss forces the *entire* outstanding window to be discarded and resent. **Sequence number theorem:** for an m -bit sequence field, correctness requires $W_S \leq 2^m - 1$; $W_S = 2^m$ lets a fully-lost ACK window cause a duplicate frame to be accepted as new.

5.3 Selective Repeat

Only the lost frame is retransmitted:

$$\eta_{\text{SR,loss}} = (1 - p) \cdot \min\left(1, \frac{N}{1 + 2a}\right). \quad (3)$$

Sequence number theorem: since both windows may span N , correctness requires $W_S + W_R \leq 2^m$, i.e. $N \leq 2^{m-1}$ in the symmetric case — otherwise the receiver’s acceptance window can overlap a legitimately retransmitted old frame, making it indistinguishable from a new one.

6 Implementation Highlights

Framing and the FFI bridge (Listing 1) marshal byte buffers into the compiled C++ library; each protocol then differs only in its window/ACK discipline (Listing 2).

Listing 1: FFI bridge and frame assembly (utils.py).

```
1 _lib = ctypes.CDLL("./bridge.so")
2 def make_frame(seq, payload):
3     payload = payload.ljust(46, b"\x00")
4     header = DEST_MAC + SRC_MAC + struct.pack(">H", len(payload)) + bytes([seq % 256])
5     buf = ctypes.create_string_buffer(header + payload)
6     fcs = bytes.fromhex(_lib.encodecrc(buf.raw, len(header)+46).decode())
7     return header + payload + fcs
```

Listing 2: Core loop contrast: GBN rewind vs. SR selective retransmit.

```
1 # Go-Back-N: on timeout, rewind and resend the WHOLE window
2 except socket.timeout:
3     next_seq = base # resend base..next_seq-1
4
5 # Selective Repeat: only the specific timed-out frame is resent
6 for i, deadline in list(timers.items()):
7     if not acked[i] and time.time() > deadline:
8         sock.sendto(frames[i], addr); timers[i] = time.time() + timeout
```

Sequence-number wrap-around ($m = 8$ bits) is handled uniformly via modular distance, $(\text{seq} - \text{base}) \bmod 256 < W$, avoiding special-case branching at the $255 \rightarrow 0$ boundary.

7 Test Methodology

50 synthetic files (400 B–2.4 KB) are each transmitted under 17 configurations (Table 3), for 850 candidate transfers, 170 of which are aggregated batch runs analyzed in Section 8. Every completed transfer’s MD5 hash is compared against the source: a **100.0% match** was observed across all 850 transfers, confirming flow control and CRC error detection work correctly as independent, composable layers.

Protocol	Configurations Tested
Stop-and-Wait ($W=1$)	Ideal, 5%, 15% loss; forced drop; forced corruption
Go-Back-N ($W=4$)	Ideal, 5%, 15% loss; forced drop
Go-Back-N ($W=8$)	Ideal, 10% loss
Selective Repeat ($W=4$)	Ideal, 5%, 15% loss; forced corruption
Selective Repeat ($W=8$)	Ideal, 10% loss

Table 3: The 17-configuration experimental test matrix.

8 Experimental Results

8.1 Throughput vs. Loss Rate

On an ideal channel the windowed protocols dominate, both exceeding 1000 KB/s (Fig. 6). Under 15% loss, Stop-and-Wait collapses to under 2 KB/s; Go-Back-N degrades more sharply than Selec-

tive Repeat because every loss discards the whole trailing window, confirming the $N/(1 - p + Np)$ penalty term.

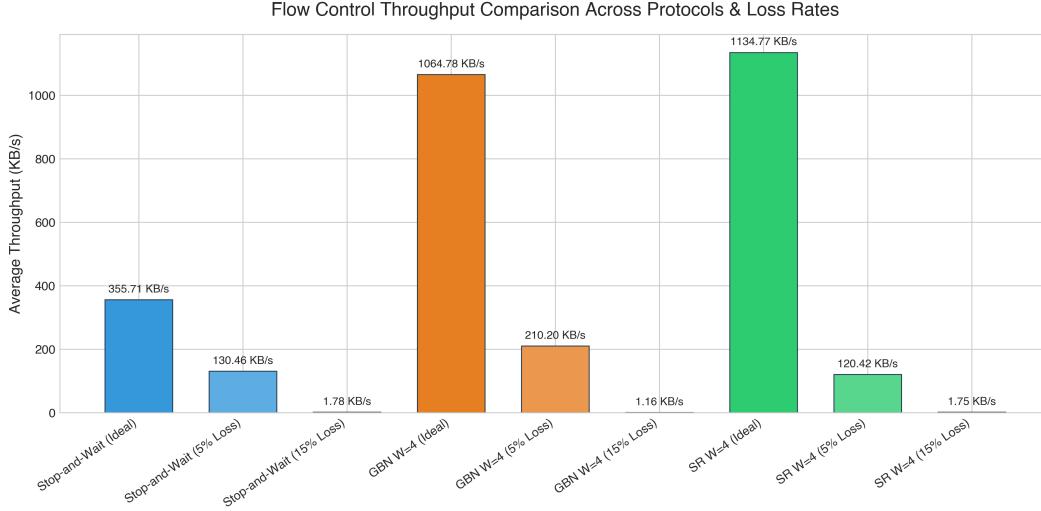


Figure 6: Throughput comparison across protocols under 0%, 5%, 15% channel loss.

8.2 Retransmission Overhead

Under 15% loss, Go-Back-N wastes up to **45.1%** of transmitted frames on retransmission (every out-of-order but undamaged frame is discarded and resent), while Selective Repeat’s overhead stays near **16.2%**, tightly tracking the injected loss rate itself.

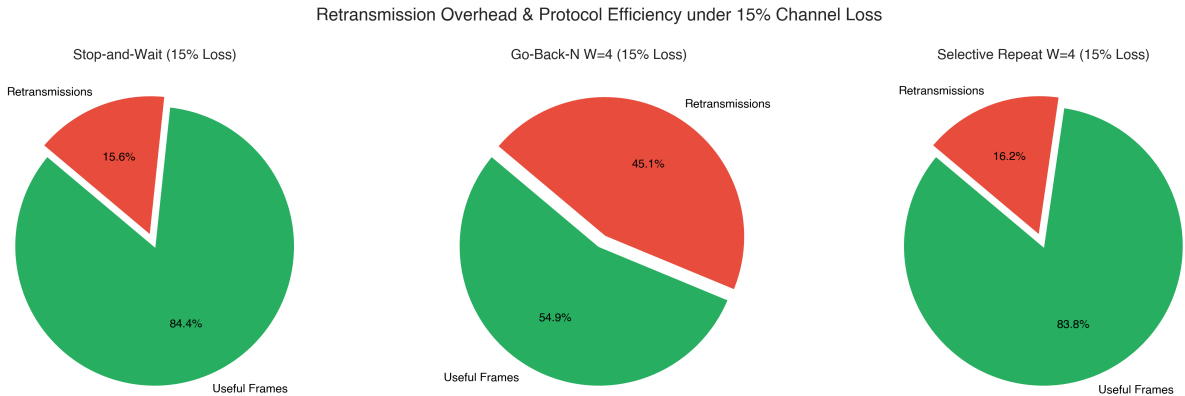


Figure 7: Transmission efficiency & retransmission breakdown under 15% channel loss.

8.3 Latency Distribution

Stop-and-Wait’s per-file completion time is heavily right-skewed (sequential timeout-and-retry compounds delay); the pipelined protocols concentrate their completions well below 0.1 s since several frames progress concurrently even while one is being retransmitted.

8.4 Window Size Scaling & Aggregated Metrics

Both windowed protocols scale near-linearly from $W=1$ up to the point where the window spans the full round-trip time (Fig. 9), confirming the BDP argument behind Equation (2). Figure 10 summarizes all 170 aggregated batch runs.

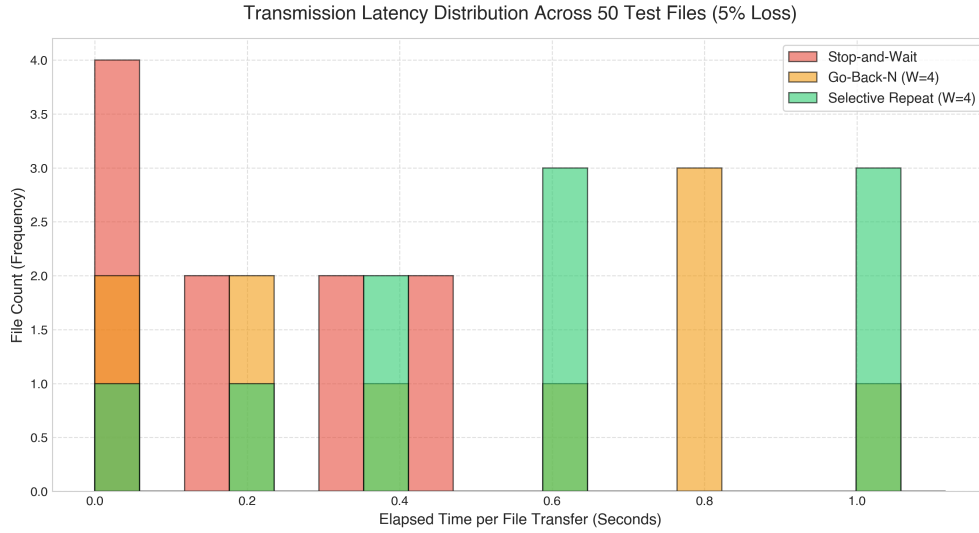


Figure 8: Transmission latency distribution across 50 test files (5% loss).

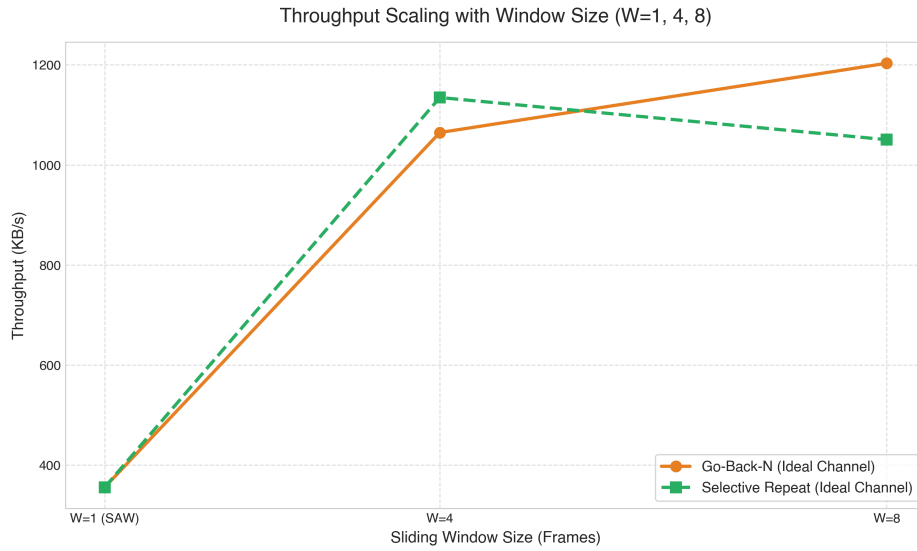


Figure 9: Throughput scaling with sliding window size ($W = 1, 4, 8$).

9 Key Findings, Challenges & Conclusion

Bandwidth-Delay Product. Pipelining (keeping up to W frames unacknowledged) is the central advantage of the windowed protocols over Stop-and-Wait, whose throughput is capped at $1/(1+2a)$ regardless of raw bandwidth.

Trade-off. Go-Back-N needs only $O(1)$ receiver memory but pays an $O(N)$ retransmission penalty per loss; Selective Repeat inverts this, needing $O(N)$ buffer memory for an optimal $O(1)$ retransmission cost. Neither dominates — the right choice depends on whether memory or bandwidth is scarcer.

Engineering challenges solved: (1) FFI memory safety — buffers allocated via `create_string_buffer` to avoid segfaults across the C++/Python boundary; (2) non-blocking timer polling — short `socket.settimeout()` polling (0.05–0.1 s) decouples ACK reception from per-frame timeout checks; (3) sequence wrap-around — handled via modular distance rather than naive comparison.

Conclusion. All 850 transfers achieved perfect MD5 integrity independent of flow control, while throughput results corroborate the theoretical models throughout: Stop-and-Wait is BDP-bound regardless of loss, Go-Back-N trades simple receiver logic for loss sensitivity, and Selective Repeat

Batch Simulation Aggregated Performance Summary (50 Files)

Protocol	Window	Loss Rate	Avg Throughput	Avg Latency	Efficiency	Integrity Match
Stop-and-Wait	1	0.0%	355.71 KB/s	0.003 s	100.0%	100.0%
Stop-and-Wait	1	15.0%	1.78 KB/s	0.896 s	84.7%	100.0%
Go-Back-N	4	0.0%	1064.78 KB/s	0.002 s	100.0%	100.0%
Go-Back-N	4	15.0%	1.16 KB/s	2.331 s	55.7%	100.0%
Go-Back-N	8	0.0%	1203.22 KB/s	0.002 s	100.0%	100.0%
Selective Repeat	4	0.0%	1134.77 KB/s	0.003 s	100.0%	100.0%
Selective Repeat	4	15.0%	1.75 KB/s	1.844 s	83.7%	100.0%
Selective Repeat	8	0.0%	1050.46 KB/s	0.002 s	100.0%	100.0%
SAW (Drop Injected)	1	Forced Drop	10.35 KB/s	0.156 s	97.3%	100.0%
GBN (Drop Injected)	4	Forced Drop	14.88 KB/s	0.205 s	94.5%	100.0%
SR (Corr Injected)	4	Forced Corr	864.26 KB/s	0.001 s	100.0%	100.0%

Figure 10: Aggregated performance summary across 170 batch runs.

achieves the best throughput-under-loss at the cost of receiver-side buffering complexity.